

TASKTRACKER

Mapeamento Arquitetural

Git e GitHub

Etapa 1 · Entrega Inicial

Pedro Rebello Borges de Barros · Ciência de Dados e Machine Learning, Turma A · Bootcamp II

```
task-tracker-bootcamp2/
├── README.md
├── LICENSE
├── .gitignore
├── especificacao/           ETAPA 1
│   ├── planejamento-logico.docx
│   ├── mapeamento-arquitetural.pptx
│   ├── roteiro-video.md
│   └── entrega-etapa-inicial.md
├── implementacao/          ETAPA 2
│   ├── requirements.txt
│   ├── src/
│   │   ├── app.py
│   │   ├── configuracao.py
│   │   ├── modelos/
│   │   │   ├── tarefa.py
│   │   │   └── usuario.py
│   │   └── servicos/
│   │       ├── gerenciador_tarefas.py
│   │       ├── gerenciador_usuarios.py
│   │       ├── validacoes.py
│   │       └── relatorio.py
```

```
├── repositorio/
│   └── repositorio_json.py
├── templates/
│   ├── base.html
│   ├── login.html
│   ├── cadastro_usuario.html
│   ├── painel.html
│   ├── formulario_tarefa.html
│   ├── confirmar_exclusao.html
│   ├── relatorio.html
│   └── admin.html
├── estaticos/
│   └── estilo.css
├── dados/
│   ├── tarefas.json
│   └── usuarios.json
├── tests/
│   ├── test_validacoes.py
│   ├── test_gerenciador_tarefas.py
│   └── test_autenticacao.py
└── docker/                  ETAPA 3
    ├── Dockerfile
    ├── docker-compose.yml
    └── .dockerignore
```

ETAPA 1 • PLANEJAR

especificacao/

Planejamento lógico, algoritmo em português estruturado, roteiro dos slides e do vídeo, e o protocolo de entrega. O código diz o quê; a especificação diz por quê.

ETAPA 2 • CONSTRUIR

implementacao/

Código-fonte em camadas, os arquivos JSON de dados e os testes automatizados. Mantém a raiz limpa e separa documento de programa.

ETAPA 3 • DISTRIBUIR

docker/

Dockerfile, compose e dockerignore. Com o objetivo de rodar o sistema com um comando só, sem precisar ter Python instalado.

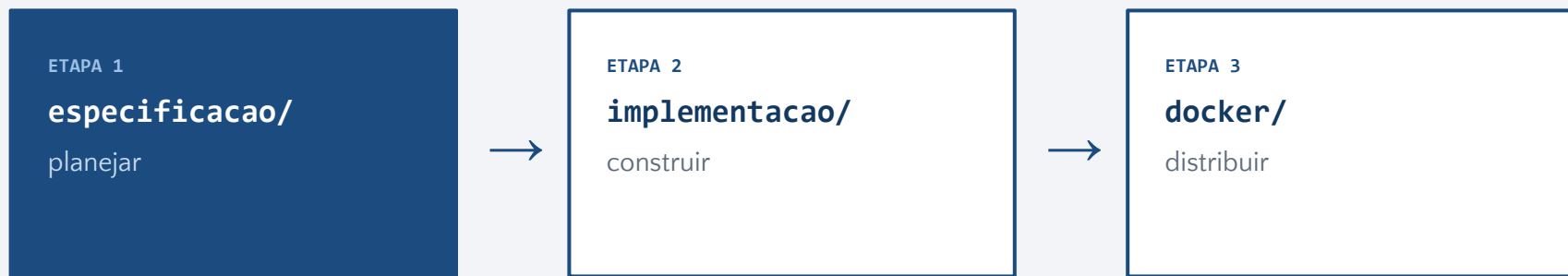
Na raiz ficam três arquivos: README.md (porta de entrada — descrição, instalação, regras e link do vídeo), LICENSE (MIT — sem ela, o código publicado fica juridicamente ambíguo) e .gitignore (venv, __pycache__, configurações do editor e qualquer arquivo com dado real ou senha).

app.py	Ponto de entrada. Cria o servidor Flask e define os endereços. Coordena — não contém regra de negócio.
configuracao.py	Caminhos dos arquivos, listas fechadas de prioridade e categoria, limite de tentativas, admin padrão.
modelos/	Classes Tarefa e Usuario. Tradução direta do modelo de dados da Etapa 1.
servicos/	AS REGRAS DE NEGÓCIO. validacoes.py traz RN01 a RN07; os gerenciadores executam as operações; relatorio.py calcula o resumo.
repositorio/	Único ponto que toca o disco. Trocar JSON por SQLite = mexer só aqui.
templates/	As telas em Jinja2 — seletor de calendário e seletores de lista fechada. Sem regra de negócio.
dados/ • tests/	Os JSON de persistência (volume do Docker na Fase 3) e os testes pytest que vão provar as sete regras.

Será dividido por responsabilidade, não por tipo de arquivo — as regras ficarão em `servicos/`, nunca dentro das telas. É isso que vai permitir testá-las sem abrir o navegador.

commit	Uma fotografia do projeto num instante, com uma mensagem que explica o que mudou e por quê.
git add	A área de stage: escolher deliberadamente o que entra em cada commit.
branches	Uma branch por funcionalidade; a main guarda só versões que funcionam.
merge e Pull Request	Integrar de forma revisada — mesmo sozinho, o PR obriga a reler o próprio código antes.
push e pull	Publicar o projeto no GitHub, trazer o que está lá e ler o histórico de versões.
conflitos	Reconhecer os marcadores e resolvê-los. Conflito não é erro: é o Git pedindo uma decisão humana.
.gitignore	Desde o primeiro commit: venv, arquivos gerados e dados reais nunca entram no repositório.

Na Fase 2 o repositório deixa de ser um lugar onde eu guardo arquivos e passa a ser o registro de como o sistema foi construído — e é isso que eu espero aprender a fazer direito.



Cada pasta-raiz corresponde a uma etapa da disciplina, na ordem em que elas acontecem. Quem abrir o repositório em qualquer momento vê em que ponto o projeto está só olhando quais pastas já têm conteúdo.

E cada arquivo em `implementacao/src/` tem contrapartida direta no planejamento: `modelos/` vem do modelo de dados, `validacoes.py` vem das sete regras de negócio, e `gerenciador_tarefas.py` vem dos passos 2 a 8 do algoritmo. O documento da Etapa 1 não é burocracia — é o mapa que a Etapa 2 vai seguir.